CS527 Lab 8 - OpenMP: Arrays and Matrices

Five C++ programs. Each one has a serial version and one or more OpenMP versions of the same function, checks that the parallel results match the serial result, and prints a timing table.

Task	File	What it does
1	`src/task1_search.cpp`	Array search: `reduction(min)`, early exit (any index), early exit (first index)
2	`src/task2_sort.cpp`	Odd-even transposition sort (parallel bubble sort) and merge sort with OpenMP tasks
3	`src/task3_histogram.cpp`	Histogram of values 0-9: naive (broken), `critical`, `atomic`, local histograms, array `reduction`
4	`src/task4_transpose.cpp`	Matrix transpose: naive and tiled
5	`src/task5_conv2d.cpp`	2D convolution with a 3x3 averaging filter
extra	`src/barrier_cost.cpp`	Measures the cost of one OpenMP barrier (used in Task 2)

The exact program output that the tables below are taken from is saved in the `results/` folder.

Build and run

```
make            (on Windows with MinGW: mingw32-make)
./task1
./task2
./task3
./task4
./task5
```

or compile one file by hand:

```
g++ -fopenmp -O2 -Wall src/task1_search.cpp -o task1
```

All five files compile with `-fopenmp -O2 -Wall` without any warning. The programs take no arguments; the sizes and thread counts are written in `main`. The handout does not give sizes, so I picked sizes big enough that one call takes at least about a millisecond.

My setup

- CPU: AMD Ryzen 7 5800H, 8 cores / 16 logical CPUs, 16 MB L3 cache
- RAM: 16 GB (2 x 8 GB DDR4-3200)

- OS: Windows 11, compiler: g++ 13.2.0 (MinGW-w64), OpenMP 4.5
- **Thread count method:** `omp_set_num_threads(n)` in the code. Each program loops over 1, 2, 4, 8 and 16 threads (16 = all logical CPUs).

How I measured time

- Time is measured with `omp_get_wtime()`.
- On my machine `omp_get_wtime()` only changes every 1 ms, so every fast function is called `repeat` times in a loop and I report the **average time of one call**. Very slow versions (`critical`, `atomic`, the O(n^2) sorts) are run only once.
- Before timing the serial version, it is called once without timing (warm-up). Without this, the first serial measurement in Task 1 came out about 1.6x too slow.
- Speedup = `T_serial / T_parallel`. Efficiency = speedup / threads.
- On Windows `long` is 32 bits, so where `n * thread_id` could overflow I compute it in `long long`.
- All numbers are from one run on a laptop that was also running other things, so they are noisy (about +-20% from run to run, as in Lab 7). For example, in Task 1 I re-timed the "first-early" search at N = 10^7 with a separate high-resolution test and got 0.59-0.64 ms for 8 threads, while the saved run shows 1.00 ms. Small differences in the tables should not be read as real.

Task 1 - Array Search

Three parallel versions; all return `-1` when the key is not in the array.

1. `reduction(min:first)` - every thread keeps the smallest matching index it has seen; OpenMP takes the minimum at the end. Gives the **first** index. A `break` is not allowed inside `omp for`, so it always scans the whole array.
2. **Early exit, any index** - the array is split into one contiguous part per thread. Every 4096 elements a thread reads a shared `found_idx` (`omp atomic read`) and stops if somebody has already found the key. The finder writes the index with `omp atomic write`. Gives **some** index where the key is.
3. **Early exit, first index** - same split, but a thread may only stop when a thread with a *lower* part has found the key (then nothing in my part can be first). The shared `best` is updated inside a `critical` (at most once per thread). Gives the **first** index.

Correctness. 2000 small random arrays (n = 1..5000, values 0..49 so keys repeat, keys 50..59 never present), different thread counts: **2000 / 2000 passed** (versions 1 and 3 must equal the serial first index; version 2 must return a position that really holds the key). Every timed call below was also checked

against the serial answer: **all correct**.

Test data: random values 0..999999, the key `-1` placed at N/3 and again at 2N/3 (so the first index is N/3), and the key `-2` which is absent.

Time in ms (speedup in brackets).

N = 10^7, key present (serial 1.00 ms, stops at index 3333333)

Threads	reduction	early, any	early, first
1	6.30 (0.16)	1.06 (0.94)	1.03 (0.97)
2	3.03 (0.33)	0.59 (1.69)	1.34 (0.75)
4	1.84 (0.54)	0.38 (2.63)	1.02 (0.98)
8	1.60 (0.63)	0.23 (4.35)	1.00 (1.00)
16	1.93 (0.52)	0.31 (3.23)	1.00 (1.00)

N = 10^7, key absent (serial 4.56 ms)

Threads	reduction	early, any	early, first
1	8.06 (0.57)	4.65 (0.98)	4.02 (1.13)
2	4.87 (0.94)	2.77 (1.65)	2.83 (1.61)
4	2.87 (1.59)	1.77 (2.58)	1.75 (2.61)
8	1.76 (2.59)	1.17 (3.90)	1.17 (3.90)
16	1.45 (3.14)	1.32 (3.45)	1.35 (3.38)

N = 10^8, key present (serial 12.2 ms)

Threads	reduction	early, any	early, first
1	65.4 (0.19)	11.9 (1.03)	12.3 (0.99)
2	42.7 (0.29)	8.1 (1.51)	14.9 (0.82)
4	25.2 (0.48)	4.2 (2.90)	10.5 (1.16)
8	15.5 (0.79)	4.0 (3.05)	7.6 (1.61)
16	13.2 (0.92)	4.3 (2.84)	6.8 (1.79)

N = 10^8, key absent (serial 38.0 ms)

Threads	reduction	early, any	early, first
1	75.2 (0.51)	44.8 (0.85)	44.2 (0.86)
2	41.4 (0.92)	24.0 (1.58)	22.8 (1.67)
4	25.2 (1.51)	14.3 (2.66)	16.1 (2.36)
8	14.6 (2.60)	11.5 (3.30)	11.0 (3.45)

Threads	reduction	early, any	early, first
16	13.2 (2.88)	13.0 (2.92)	13.8 (2.75)

Explanation.

- **reduction** never beats serial when the key is present: serial stops at N/3 but the reduction always reads all N elements, three times as much work. When the key is absent both read everything, and the reduction reaches about 3x with 16 threads. With 1 thread it is about 1.8x slower than serial. I looked at the assembly: the inner loops are almost the same instructions, but the reduction loop makes two jumps per element and the serial loop only one, so the slowdown comes from how the compiler arranged the loop, not from OpenMP.
- **Early exit, any index** is the fastest: about 4x with 8 threads. When the key is present the threads stop as soon as the first one finds it. It does not keep going past about 3-4x because a search does almost no work per element and the threads are waiting for memory: for N = 10^8 (400 MB) absent, 11.5 ms means about 35 GB/s, close to what this RAM can deliver.
- **Early exit, first index** is the interesting one. To be sure that N/3 is the *first* match, every position before N/3 must still be checked. With 2 threads, thread 0 owns `[0, N/2)` and has to scan all the way to N/3 by itself, the same as serial, so there is no speedup (0.75-0.82). With p threads the thread that owns position 0 always has to scan its whole part (N/p), so in the best case the time is N/p instead of N/3. In addition, the threads to the right of the key keep scanning until a lower thread has found it. In my instrumented test (N = 10^7, 8 threads) the threads read 6.7 million elements in total against 3.3 million for serial, and since the search is limited by memory speed, those extra reads cost time. So it gets only 1.6-1.8x for N = 10^8. When the key is absent nobody can stop early, and versions 2 and 3 are the same.

Synchronization used. Version 1 needs nothing beyond the `reduction` clause (each thread has a private copy of `first`). Versions 2 and 3 share one variable between all threads, so every read and write of it is `omp atomic`, otherwise it would be a data race. In version 3 the update "`if (my_idx < best) best = my_idx`" is a read followed by a write and must not be split, so it is inside a `critical`. This is cheap because each thread enters it at most once.

Why not just `break`? OpenMP does not allow `break` out of an `omp for` loop, because the other threads would not know the loop has ended. OpenMP 4.0 has `#pragma omp cancel for`, but it only works when the program is started with the environment variable `OMP_CANCELLATION=true`, so I split the array by hand instead.

Task 2 - Array Sorting

Part A: parallel bubble sort (odd-even transposition sort)

Bubble sort as written normally cannot be parallelized: the element moved in step `i` is compared again in step `i+1`. **Odd-even transposition sort** is the parallel form of bubble sort. It runs `n` phases:

- even phase: compare and swap the pairs (0,1), (2,3), (4,5), ...
- odd phase: compare and swap the pairs (1,2), (3,4), (5,6), ...

Within one phase the pairs do not overlap, so they can all be done at the same time. After `n` phases the array is sorted.

```
#pragma omp parallel
for (long phase = 0; phase < n; phase++) {
    #pragma omp for
    for (long i = phase % 2; i < n - 1; i += 2)
        if (a[i] > a[i + 1]) swap(a[i], a[i + 1]);
    // implicit barrier
}
```

Synchronization needed:

1. *Inside a phase - none.* Every element belongs to at most one pair, so no two threads ever touch the same element in the same phase.
2. *Between phases - a barrier.* The odd phase compares `(a[1], a[2])`, but `a[1]` was just changed by the even pair `(0,1)` and `a[2]` by `(2,3)`. If a thread started the odd phase while its neighbour was still in the even phase, it could read a value before the neighbour's swap and the result would be wrong. The implicit barrier at the end of `omp for` makes every thread finish the phase before anyone starts the next one.
3. The parallel region is opened **once**, outside the phase loop, so the threads are created once instead of `n` times. Each thread runs the `phase` loop itself (`phase` is private), and all of them go through the same phases in the same order, which is required because every thread must reach the same `omp for`.

Correctness: every result was compared element by element with `std::sort` on the same input: **all correct**. Each sort was run once.

Time in ms (speedup against the serial odd-even sort in brackets):

Threads	N = 10^4	N = 5x10^4	N = 10^5
serial bubble sort	240	6093	25680
serial odd-even sort	57	1298	7300
1	58 (0.98)	1111 (1.17)	6777 (1.08)
2	205 (0.28)	1661 (0.78)	6391 (1.14)
4	377 (0.15)	1583 (0.82)	4463 (1.64)

Threads	N = 10^4	N = 5x10^4	N = 10^5
8	560 (0.10)	3110 (0.42)	5446 (1.34)
16	1020 (0.06)	6673 (0.19)	12892 (0.57)

Explanation. The parallel version only helps a little, and only for the largest array (1.64x with 4 threads at N = 10^5). For N = 10^4 it gets *slower* with every thread added. The reason is the barrier after every phase: there are `n` phases, so `n` barriers. To check this I measured the cost of one barrier on my machine (`src/barrier_cost.cpp`, output in `results/barrier_cost_output.txt`):

Threads	1	2	4	8	16
one barrier (us)	0.00	13.4	26.3	50.5	137.6

One phase for N = 10^4 is only about 5.7 us of work (57 ms / 10^4 phases), while one barrier with 16 threads costs about 138 us. So 10^4 barriers alone take about 1.4 s, which explains the 1020 ms measured (the work part is negligible). For N = 10^5 a phase is about 73 us of work, so with 4 threads (about 18 us of work + 26 us of barrier per phase) there is finally a small gain. With 8 and 16 threads the barrier cost grows faster than the work per thread shrinks. These barriers are expensive on this Windows / MinGW setup (this matches Lab 7, where starting a parallel region cost 4-67 us). I also ran the whole program with `OMP_WAIT_POLICY=active` (threads spin instead of sleeping at the barrier), and the times were the same within noise, so the cost is not from threads going to sleep and waking up.

The serial bubble sort is 3.5-4.7x slower than the serial odd-even sort, although both do about n^2/2 comparisons. I think this is because in bubble sort each comparison uses the result of the previous swap (`a[i+1]` is compared again in the next step), so the CPU has to wait for each step, while the pairs in one odd-even phase are independent and the CPU can work on several at once. That is why I compare the parallel version against the serial *odd-even* sort, which is the same algorithm.

Part B: merge sort with OpenMP tasks

Merge sort is a much better fit for parallel execution: the two halves are completely independent.

```
void merge_sort_tasks(int *a, int *tmp, long lo, long hi) {
    if (hi - lo <= TASK_CUTOFF) { merge_sort_serial(a, tmp, lo, hi); return; }
    long mid = lo + (hi - lo) / 2;
    #pragma omp task
    merge_sort_tasks(a, tmp, lo, mid);
    #pragma omp task
```

```
    merge_sort_tasks(a, tmp, mid, hi);
    #pragma omp taskwait
    merge(a, tmp, lo, mid, hi);
}
// called from: #pragma omp parallel  ->  #pragma omp single
```

Synchronization needed:

1. `#pragma omp single` - only one thread starts the recursion; the other threads of the team pick up the tasks it creates.
2. `#pragma omp taskwait` - the merge reads both halves, so it must wait until both child tasks have finished sorting them. Without it the merge could read a half that is still being sorted.
3. No locks: the two halves use different parts of `a` and of the helper array `tmp`, so the tasks never write the same memory.
4. Below `TASK_CUTOFF` = 10000 elements no new tasks are made (making a task costs more than sorting a few thousand numbers).

Correctness: every sort compared with `std::sort`: **all correct**.

Time in ms (S = speedup against serial merge sort, E = efficiency):

Threads	N = 10^6	S	E	N = 10^7	S	E
serial merge sort	69.0			866.8		
1	74.6	0.93	0.93	841.6	1.03	1.03
2	41.6	1.66	0.83	460.2	1.88	0.94
4	29.0	2.38	0.60	342.6	2.53	0.63
8	19.6	3.52	0.44	232.8	3.72	0.47
16	14.6	4.71	0.29	163.8	5.29	0.33

(`std::sort`, one thread, for reference: 54 ms and 640 ms.)

Explanation. Merge sort keeps improving up to 16 threads (5.3x for N = 10^7), much better than odd-even sort, and it does O(n log n) work instead of O(n^2). It does not reach 16x because the **last merge is serial**: merging the two final halves (10^7 elements) is done by one thread while the others have nothing to do, and the level below it has only 2 merges, then 4, and so on. So the top few levels cannot use all 16 threads. Merging also mostly moves data, so it is partly limited by memory speed.

Odd-even vs merge sort. For N = 10^5 the best odd-even time is 4.5 s; merge sort sorts 100 times more data (10^7) in 0.16 s. Choosing a better algorithm matters much more than adding threads.

Task 3 - Histogram (values 0..9)

The array has random values 0..9. Versions:

0. **naive** (`parallel for` + `hist[a[i]]++`) - wrong, data race
1. `critical` around every `hist[a[i]]++`
2. `atomic` on every `hist[a[i]]++`
3. **local histograms** - each thread counts into its own private array `my_hist[10]` (declared inside the parallel region), then adds it into the shared histogram once inside a `critical`
4. **array reduction** `reduction(+:hist[:10])` (OpenMP 4.5) - OpenMP makes the private copies and adds them up for me

Correctness. Every result was compared bin by bin with the serial histogram. The serial counts add up to exactly N. Versions 1-4 matched for every thread count. The naive version is wrong every time:

```
Naive parallel version, N = 10^7, 8 threads, 5 runs:
Run 1: bin 0 =    511816 (correct 1000853), sum of counts =   5010987   WRONG
Run 2: bin 0 =    505696 (correct 1000853), sum of counts =   4937221   WRONG
Run 3: bin 0 =    501585 (correct 1000853), sum of counts =   4895034   WRONG
Run 4: bin 0 =    505468 (correct 1000853), sum of counts =   4941506   WRONG
Run 5: bin 0 =    498089 (correct 1000853), sum of counts =   4858945   WRONG
```

About **half of all increments are lost**, and the result is different each run. `hist[a[i]]++` is read - add - write; with only 10 bins, two threads very often update the same bin at the same time, both read the same old count, and one increment disappears.

Time in ms (speedup in brackets). `atomic` and `critical` were run once.

N = 10^7 (serial 5.39 ms)

Threads	reduction	local	atomic	critical
1	4.39 (1.23)	5.46 (0.99)	21	43
2	2.46 (2.19)	3.13 (1.72)	96	95
4	1.49 (3.62)	1.81 (2.98)	120	173
8	1.36 (3.96)	1.86 (2.90)	112	237
16	1.24 (4.35)	1.30 (4.15)	109	286

N = 10^8 (serial 61.9 ms)

Threads	reduction	local	atomic	critical
1	46.6 (1.33)	60.8 (1.02)	218	not run
2	34.0 (1.82)	42.8 (1.45)	639	not run
4	17.9 (3.46)	27.7 (2.23)	1155	not run
8	11.0 (5.63)	14.5 (4.27)	1113	not run

Threads	reduction	local	atomic	critical
16	12.8 (4.84)	13.2 (4.69)	1091	not run

Explanation.

- **`atomic` and `critical` are correct but much slower than serial**, and get slower when threads are added (atomic: 3.5x slower than serial with 1 thread, 18x slower with 4 threads at N = 10^8). Every single element needs an atomic operation or a lock. Worse, the whole histogram is 10 x 4 bytes = 40 bytes, which is **one 64-byte cache line**, so every thread is fighting for the same cache line all the time and it keeps moving between cores. `critical` is slower still because it is a real lock (one lock for all 10 bins), not a single CPU instruction.
- **Local histograms and `reduction` are correct and fast**: each thread counts in its own private array with no sharing at all, and the private results are combined only once per thread (10 additions). The two are the same idea; `reduction` is just shorter to write. They reach about 4-5.6x. Beyond 8 threads there is no gain, as the threads are again limited by reading the 400 MB array from memory.
- The `reduction` version is 1.2-1.3x faster than serial even with 1 thread. I checked the assembly: the inner loops of the serial and the reduction version are the same 5 instructions, so this is not a difference in work, and I could not find the cause (possibly where the count array sits in memory). Measured against its own 1-thread time, `reduction` scales by 3.2x (N = 10^7) and 4.2x (N = 10^8) with 8 threads.

Preventing race conditions - summary. A private copy per thread (`reduction` or a manual local array) is the right method: no lock is needed during counting, only one short combine step at the end. `atomic` and `critical` also prevent the race but turn the counting back into a one-at-a-time operation.

Task 4 - Matrix Transpose

Storage: 2D arrays of `double` (an array of row pointers, as in Lab 7). `A[i][j] = i*N + j`, so every element is different and a transposed matrix can be checked exactly with `==` (`B[j][i] == A[i][j]` for all i, j). **All serial and parallel results passed this check.**

Example printed by the program (3x3, 4 threads):

```
A:              B = A^T:
   1   2   3       1   4   7
   4   5   6       2   5   8
   7   8   9       3   6   9
```

Two versions:

1. **naive** - `parallel for` over the rows `i` of A, inner loop over `j`, `B[j][i] = A[i][j]`. A is read along a row, but B is written down a column: each write goes to a different row of B.
2. **tiled** - the matrix is cut into 32 x 32 tiles and each tile is transposed on its own. One tile of A and one of B (8 KB each) fit in the L1 cache, so the cache lines of B that a tile writes are reused while they are still in the cache. `collapse(2)` is used on the two tile loops so there are (N/32)^2 pieces of work to share.

Synchronization: none needed. Each `B[j][i]` is written by exactly one iteration and A is only read.

N = 5, time per call in **microseconds**: serial 0.02 us, parallel 4.1 us (1 thread) up to 102 us (16 threads). Parallel is always hundreds of times slower here - the whole job is smaller than the cost of starting a parallel region.

Time in ms (speedup against the **serial naive** transpose in brackets):

N = 1024

Threads	naive	tiled
serial	1.94	1.57 (1.24)
1	1.94 (1.00)	1.57 (1.24)
2	1.52 (1.28)	1.21 (1.60)
4	0.92 (2.12)	0.69 (2.82)
8	0.55 (3.56)	0.47 (4.15)
16	0.43 (4.54)	0.38 (5.09)

N = 2048

Threads	naive	tiled
serial	12.04	7.73 (1.56)
1	12.14 (0.99)	7.91 (1.52)
2	8.99 (1.34)	6.73 (1.79)
4	6.24 (1.93)	5.53 (2.18)
8	5.35 (2.25)	5.29 (2.28)
16	5.45 (2.21)	5.33 (2.26)

N = 4096

Threads	naive	tiled
serial	53.7	36.6 (1.47)
1	53.5 (1.00)	36.6 (1.47)

Threads	naive	tiled
2	38.5 (1.40)	29.2 (1.84)
4	24.8 (2.16)	22.5 (2.38)
8	18.4 (2.93)	19.9 (2.71)
16	19.0 (2.82)	20.1 (2.68)

Explanation.

- Transpose does no arithmetic at all; it only moves data. So, like vector and matrix addition in Lab 7, it is limited by memory speed.
- **N = 1024** (8 MB per matrix, both together fit in the 16 MB L3 cache) scales best: 4.5x naive and 5.1x tiled with 16 threads.
- **N = 2048 and 4096** (32 MB and 128 MB per matrix) do not fit in the cache. The speedup stops at about 2.2x and 2.9x, and adding threads beyond 4-8 does not help.
- **Tiling helps on one thread** (1.24x-1.56x faster than naive). In the naive version every write to B goes to a different row, and with N = 4096 each row is 32 KB, so consecutive writes land on different memory pages and different cache lines; a cache line of B is loaded for one value and is usually thrown out before its other 7 values are written. Inside a tile only 32 rows of B are touched, so those lines stay in the cache.
- **With many threads the two versions become equal** for the large matrices: the memory is the bottleneck and both versions move the same amount of data.

Task 5 - 2D Convolution (3x3 averaging filter)

Storage: 2D arrays of `double` (row pointers). The image is N x N, the output is (N-2) x (N-2) because only valid positions are used (no padding). `out[i][j]` is the value centred on image pixel `(i+1, j+1)`, so `out[0][0]` is `O[1][1]` in the handout. The kernel is a 3x3 array with every weight 1/9, and the function takes any 3x3 kernel.

```
#pragma omp parallel for
for (int i = 0; i < m; i++)
    for (int j = 0; j < m; j++) {
        double sum = 0;
        for (int ki = 0; ki < 3; ki++)
            for (int kj = 0; kj < 3; kj++)
                sum += img[i + ki][j + kj] * K[ki][kj];
        out[i][j] = sum;
    }
```

(Strictly, convolution flips the kernel first. For a symmetric kernel such as the averaging one this makes no difference.)

Correctness.

1. The handout example is reproduced exactly:

```
Handout example: input image (5 x 5)
   10.00   20.00   30.00   40.00   50.00
   60.00   70.00   80.00   90.00  100.00
  110.00  120.00  130.00  140.00  150.00
  160.00  170.00  180.00  190.00  200.00
  210.00  220.00  230.00  240.00  250.00
Output (3 x 3), parallel with 4 threads:
   70.00   80.00   90.00
  120.00  130.00  140.00
  170.00  180.00  190.00
Matches the hand-computed answer (70 80 90 / 120 130 140 / 170 180 190): yes
Parallel == serial: yes
```

2. For the large random images (values 0..255), each parallel output was compared with the serial output using `==`. Each output element is computed by the same 9 additions in the same order in every version, so the results must be bit-for-bit equal. **All matched.**

Synchronization: none needed. The input image is only read, each output element is written by exactly one iteration, and `sum` is declared inside the loop so it is private to each thread.

Time in ms. S = speedup, E = efficiency.

N = 1024 (serial 5.46 ms)

Threads	rows (ms)	S	E	collapse(2) (ms)	S	E
1	5.02	1.09	1.09	6.19	0.88	0.88
2	3.67	1.49	0.74	4.03	1.36	0.68
4	2.41	2.27	0.57	2.58	2.11	0.53
8	1.36	4.01	0.50	1.43	3.81	0.48
16	0.86	6.34	0.40	0.91	5.99	0.37

N = 2048 (serial 26.1 ms)

Threads	rows (ms)	S	E	collapse(2) (ms)	S	E
1	22.8	1.15	1.15	26.3	0.99	0.99
2	17.8	1.47	0.73	20.5	1.27	0.64
4	9.9	2.64	0.66	9.5	2.74	0.68
8	5.6	4.69	0.59	5.4	4.80	0.60

Threads	rows (ms)	S	E	collapse(2) (ms)	S	E
16	4.1	6.31	0.39	4.3	6.11	0.38

N = 4096 (serial 95.5 ms)

Threads	rows (ms)	S	E	collapse(2) (ms)	S	E
1	88.1	1.08	1.08	86.8	1.10	1.10
2	52.6	1.82	0.91	59.0	1.62	0.81
4	30.6	3.12	0.78	34.8	2.74	0.69
8	18.2	5.24	0.66	20.0	4.76	0.60
16	17.2	5.55	0.35	21.0	4.54	0.28

Explanation.

- Convolution scales better than the transpose: about 4-5x with 8 threads and up to 6.3x with 16. Every output value needs 9 multiplications and 9 additions, and the 3 image rows it reads are reused by the neighbouring outputs (they stay in the cache), so there is real computation per byte loaded from memory.
- Efficiency falls with more threads (0.35-0.40 at 16). Only 8 of the 16 threads are real cores, and for N = 4096 (128 MB in, 128 MB out) the memory again starts to limit: 8 and 16 threads give almost the same time.
- **`collapse(2)` does not help.** With 1022 or more rows there are already far more rows than threads, so every thread gets an equal share without it. It is a little slower in most rows of the tables, probably because with `collapse(2)` each iteration has to compute its `i` and `j` from a single combined counter.
- The parallel version with 1 thread is 8-15% faster than the serial function, which runs the same code. As in Lab 7 I treat differences of this size as noise.